Lab 3.1: Using Resource-Based Policies to Secure an S3 Bucket — Lab Report

20 Oct 2026 · Fatma · AWS Academy Cloud Foundations (hands-on lab, AWS Console)

Signed in as the IAM user devuser, I tested what identity-based policies allow and deny in Amazon EC2, S3 and IAM. I then assumed two IAM roles to see how role policies and S3 bucket policies (resource-based policies) combine to grant access. Every denied action failed with the expected error and every allowed action succeeded. The final grade was 100/100. The first submission scored 66.67 because the Task 5 upload to bucket2 was missing, so I completed it and resubmitted (Phase 8).

Account number and bucket names are redacted in the screenshots and policy files; <ACCOUNT_ID> and <BUCKETn_NAME> are placeholders.

Phase 1 — Lab environment and sign-in (Task 1)

The lab pre-built three S3 buckets (bucket1, bucket2, bucket3), an IAM group (DeveloperGroup) with one user (devuser), and two roles. The environment was in us-east-1.

Item	Value
IAM user	devuser (member of DeveloperGroup)
Group policy	DeveloperGroupPolicy (identity-based, attached to the group)
Roles	BucketsAccessRole and OtherBucketAccessRole
Buckets	bucket1, bucket2, bucket3 (pre-built) and depi-lab3.1 (created by me in Task 4)
Region	US East (N. Virginia) us-east-1
Account ID	<ACCOUNT_ID> (redacted)

I signed in with the IAMUserLoginURL from the AWS Details panel, using the user name devuser. My first attempt used the green "AWS" link, which opens the console as a federated user (voclabs/…) and shows "Access denied" even on the account name. The lab warns against it. The correct sign-in shows devuser in the top-right corner.

Phase 2 — Attempting read-level access as devuser (Task 2)

As devuser I opened EC2 and S3 to see what the group policy allows.

Amazon EC2

The EC2 dashboard showed Access denied on every resource tile. Opening Instances returned an exception stating that devuser is not authorized to perform ec2:DescribeInstances because no identity-based policy allows it. The Launch instances wizard failed the same way on the Key pair dropdown (ec2:DescribeKeyPairs). Both errors name the missing action explicitly.

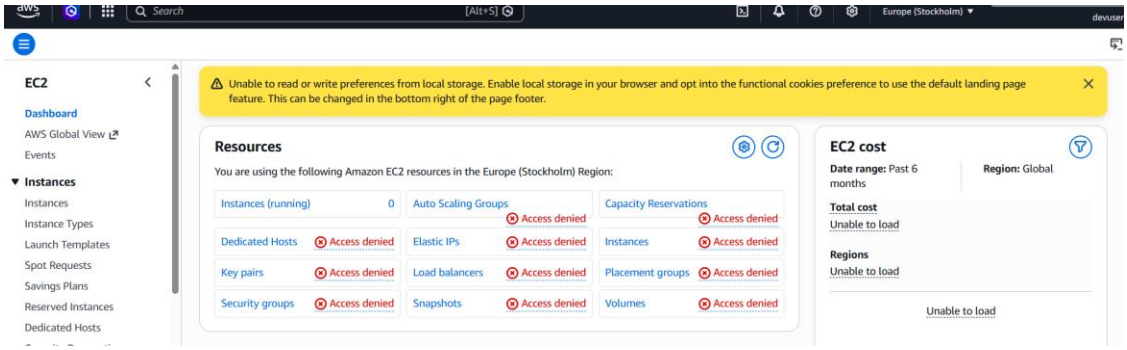


Figure 1 — EC2 dashboard as devuser: every resource shows Access denied.

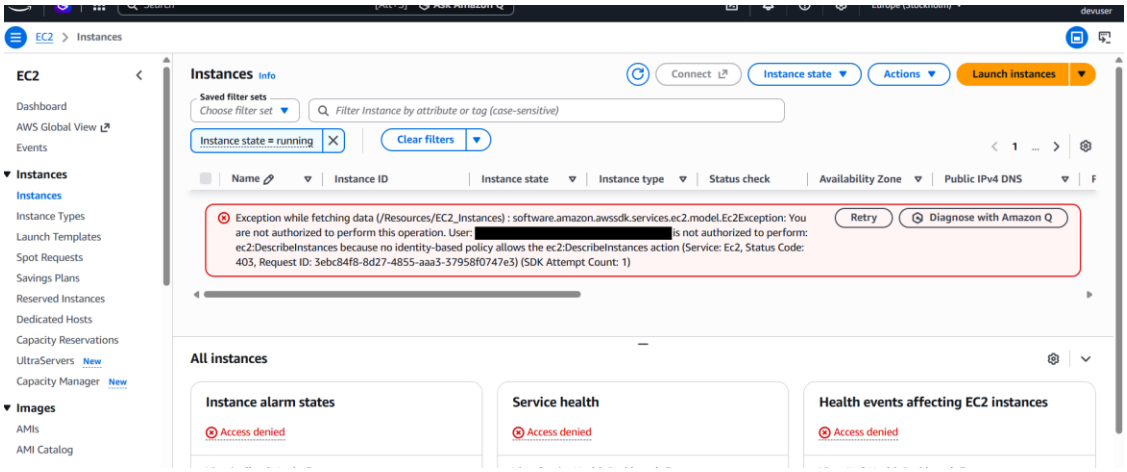


Figure 2 — Instances page: ec2:DescribeInstances is not allowed by any identity-based policy.

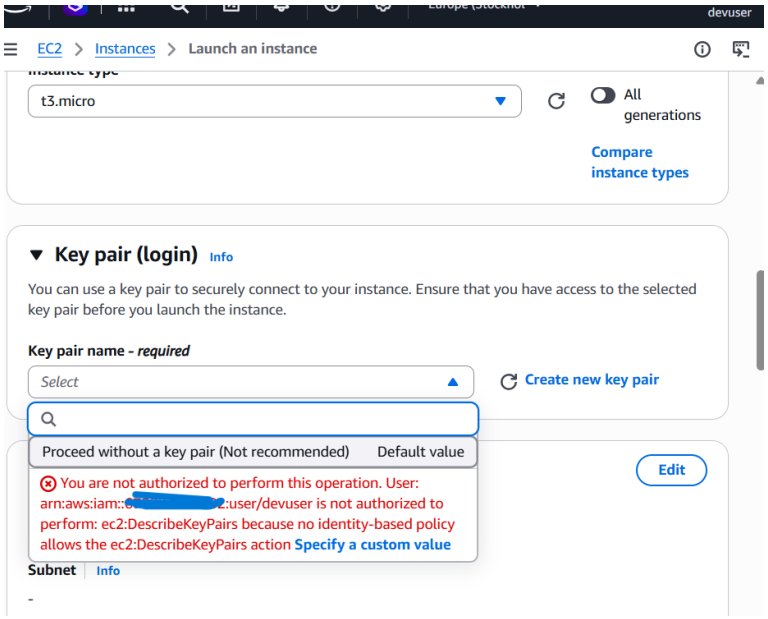


Figure 3 — Launch wizard: the Key pair dropdown fails with ec2:DescribeKeyPairs not allowed.

Amazon S3

The S3 console listed all three buckets. The newer console does not show the "Access: Insufficient permissions" column the lab text describes, so the buckets table has only Name, Region and Creation

date. Opening bucket1 also listed its two objects, Image1.jpg and Image2.jpg. This differs from the lab text, but it fits the policy: devuser has s3:ListBucket on all resources (Phase 3). Uploading a file to bucket1 failed with Access denied, so devuser can list but cannot write.

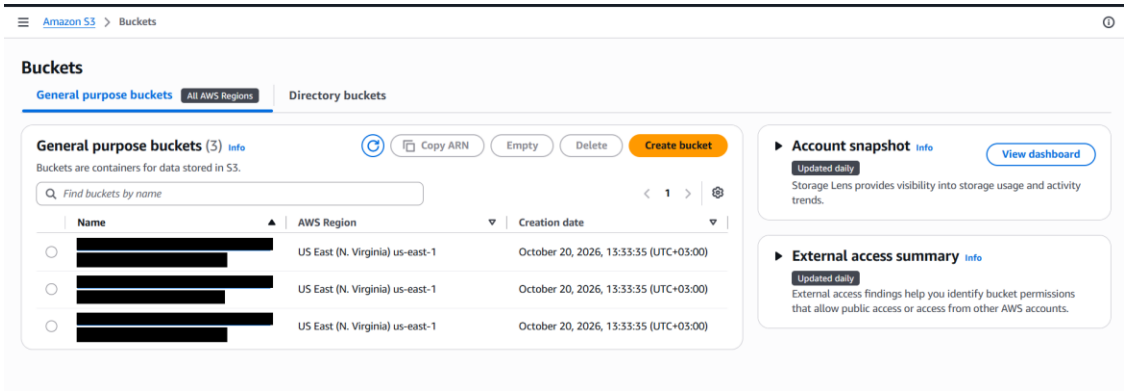


Figure 4 — Buckets list as devuser: three buckets visible, names redacted.

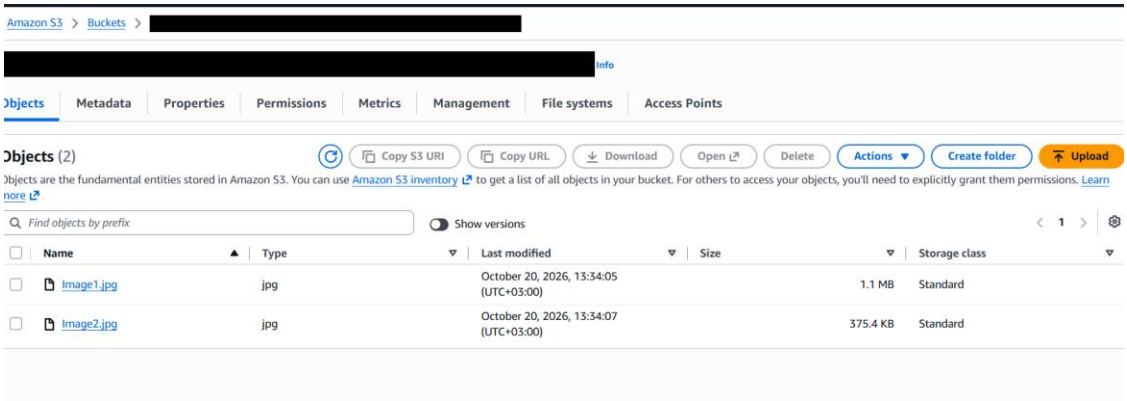


Figure 5 — bucket1 opens and lists Image1.jpg and Image2.jpg (s3:ListBucket is allowed).

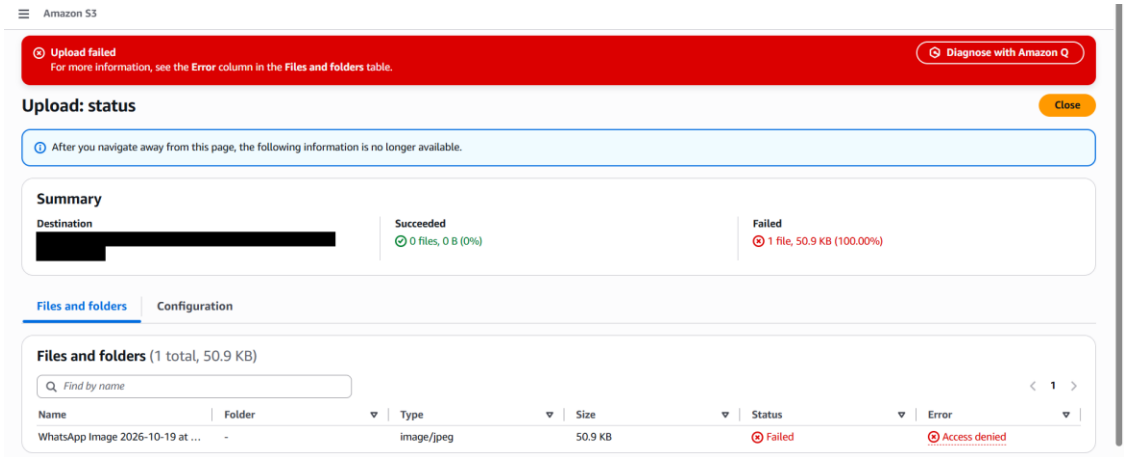


Figure 6 — Upload to bucket1 fails with Access denied (no s3:PutObject).

Phase 3 — Analysing the identity-based policy (Task 3)

In IAM > User groups > DeveloperGroup, the Users tab shows devuser as the only member. The Permissions tab shows DeveloperGroupPolicy attached to the group, so it governs everything devuser does. I copied the JSON and saved it as DeveloperGroupPolicy.json (full text in Appendix A).

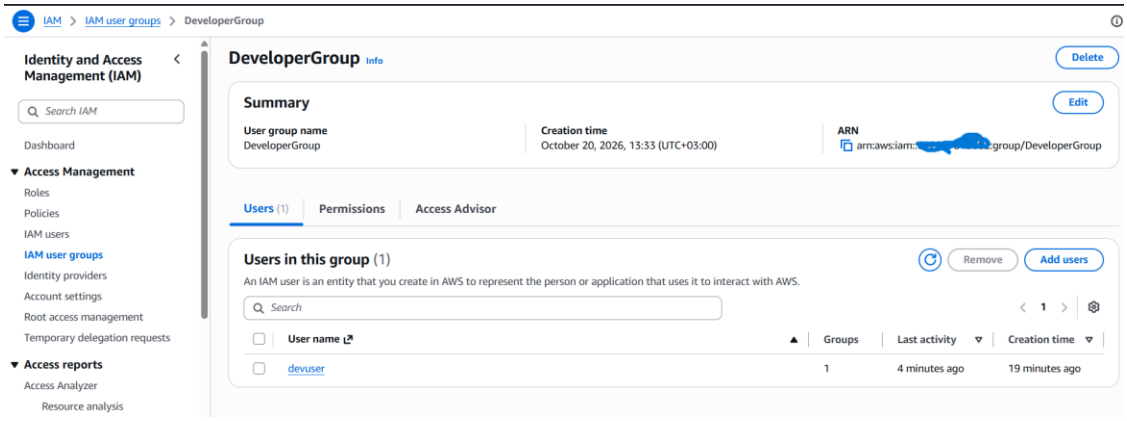


Figure 7 — DeveloperGroup: devuser is the only member.

The policy is one Allow statement on Resource "*". Reading its action list explains each result from Phase 2 and Phase 4:

What I observed	Why (from the policy)
EC2 pages all denied	No ec2:* actions are listed
IAM dashboard denied on GetAccountSummary, but group and policy pages open	iam:Get* is limited to named actions (GetGroup, GetRole, GetPolicy…), plus iam:List* and iam:Describe*; GetAccountSummary is not included
Could create a bucket	s3:CreateBucket is allowed
Could see the buckets and list bucket1	s3:ListAllMyBuckets and s3:ListBucket are allowed
Could not upload or download objects	No s3:PutObject or s3:GetObject
Could switch roles	sts:AssumeRole is allowed
Encryption warning after creating a bucket	s3:PutEncryptionConfiguration is not allowed

Phase 4 — Attempting write-level access (Task 4)

I created a bucket named depi-lab3.1 in us-east-1. The bucket was created ("Successfully created bucket"). A red banner appeared as well: devuser lacks s3:PutEncryptionConfiguration, so S3 could not apply default encryption. The bucket still exists, so this does not block the lab.

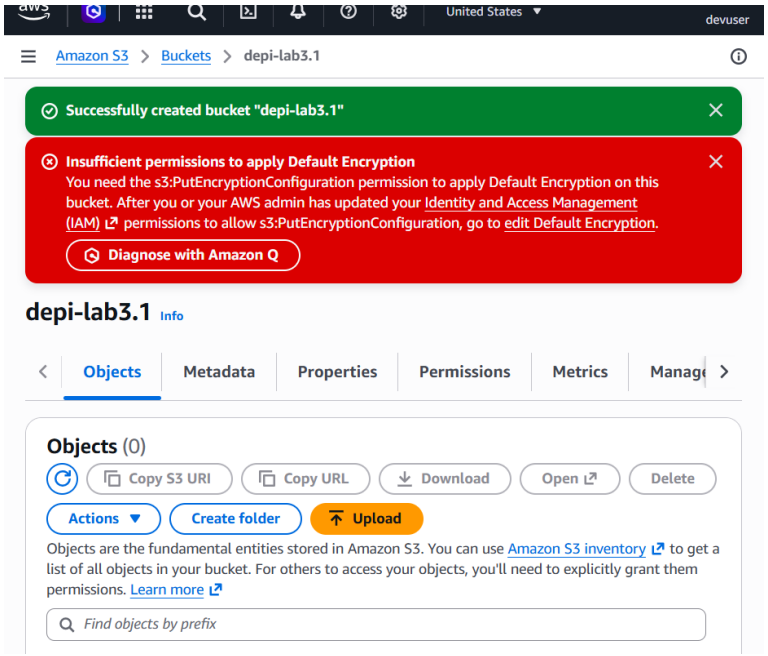


Figure 8 — Bucket depi-lab3.1 created; the default-encryption banner shows a missing permission.

I then tried to upload DeveloperGroupPolicy.json into it. The upload failed with Access denied: the policy allows s3:CreateBucket but not s3:PutObject.

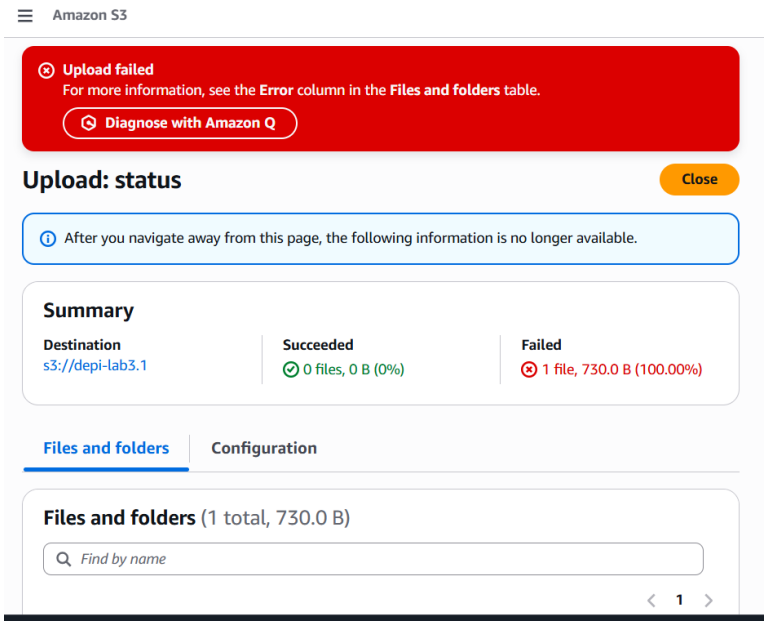


Figure 9 — Upload to depi-lab3.1 fails (0 files succeeded, 1 failed).

Phase 5 — Assuming BucketsAccessRole (Task 5)

Before the role: devuser cannot read objects

Downloading Image2.jpg from bucket1 as devuser returned an AccessDenied XML error. Its message names the cause: devuser is not authorized to perform s3:GetObject on bucket1/Image2.jpg because no identity-based policy allows that action.



Figure 10 — AccessDenied XML for s3:GetObject on bucket1/Image2.jpg.

Switch role

From the account menu I chose Switch role with the account ID and the role name BucketsAccessRole (display name left blank). The console bar then showed BucketsAccessRole @ <ACCOUNT_ID>. While the role is active, devuser's own permissions are not applied; only the role's permissions are. The S3 buckets table now showed four buckets: the role has s3:ListAllMyBuckets, so it also sees depi-lab3.1.

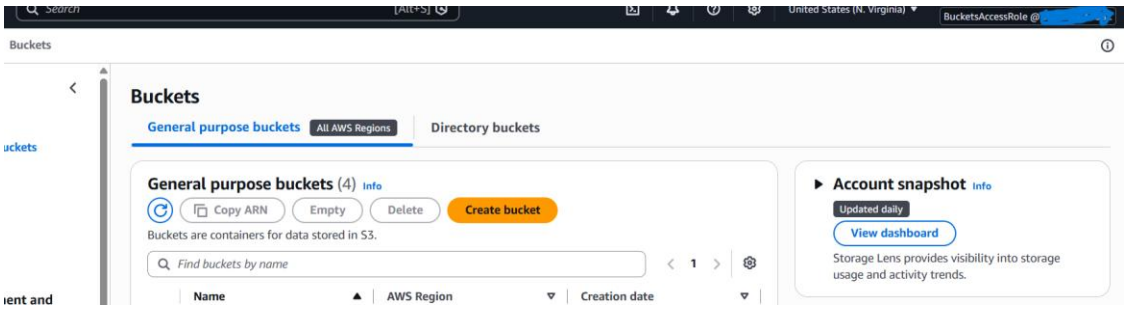


Figure 11 — Role session: bucket list shows 4 buckets.

Downloading Image2.jpg from bucket1 now succeeded.

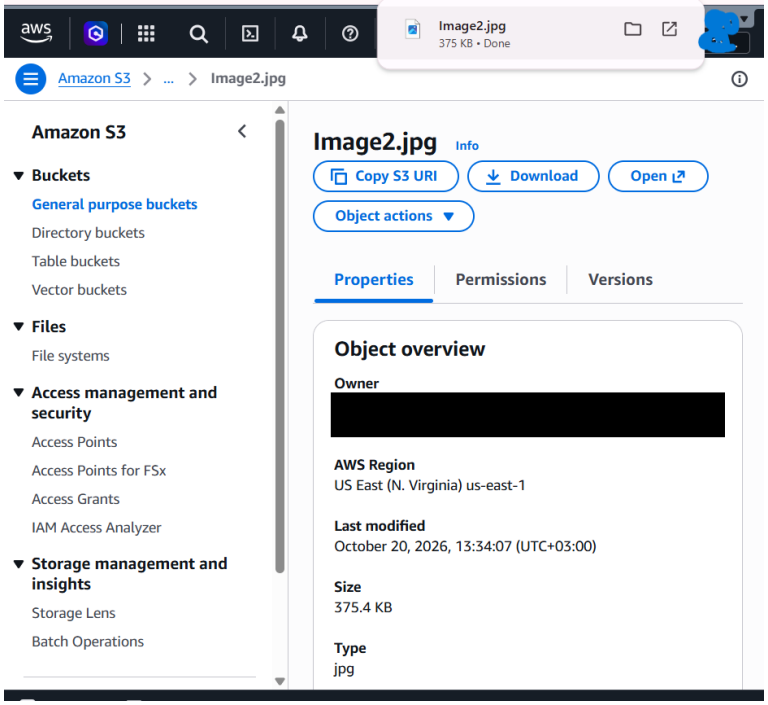


Figure 12 — Image2.jpg downloaded with BucketsAccessRole.

Role permissions are different from devuser's, not additional to them. In IAM > User groups, the role session returned "Access denied to iam:ListGroups". After Switch back, the same page listed DeveloperGroup again.

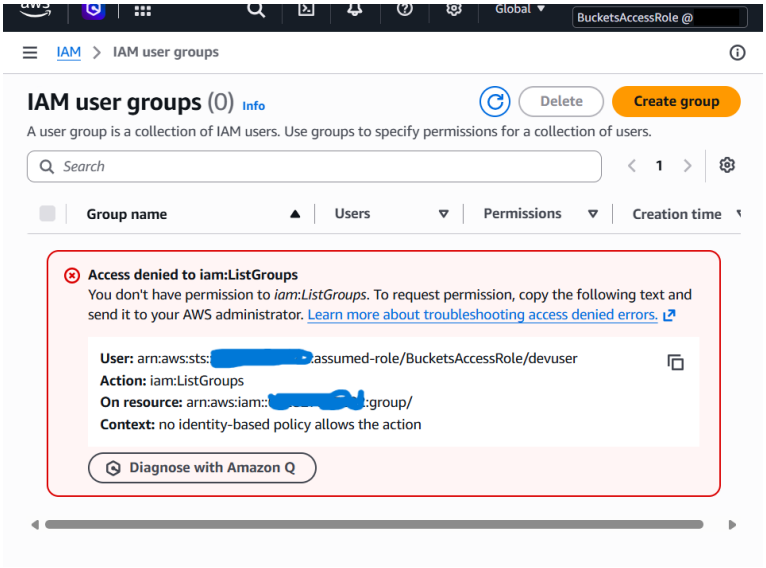


Figure 13 — As the role: iam:ListGroups is denied (the role has no IAM permissions).

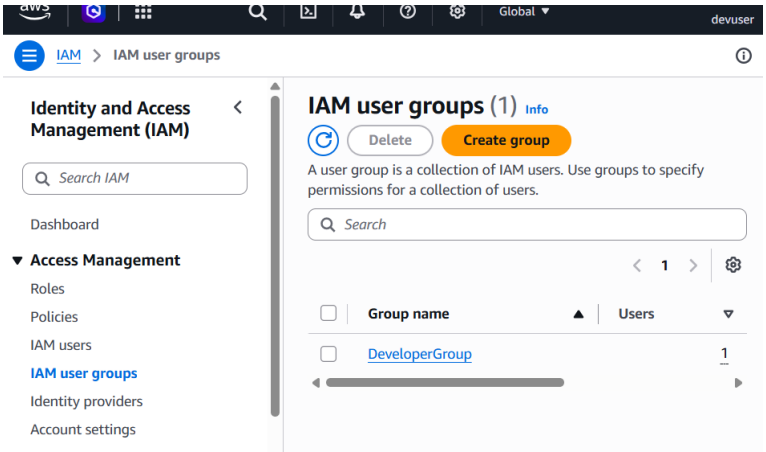


Figure 14 — After Switch back: devuser can list groups again.

The role's policy

BucketsAccessRole has two policies: ListAllBucketsPolicy (s3:ListAllMyBuckets on all resources) and GrantBucket1Access, which allows s3:GetObject, s3:ListObjects and s3:ListBucket only on bucket1 and its objects (bucket ARN plus /*). It has no s3:PutObject and does not mention bucket2. Full JSON is in Appendix B. The "/*" suffix is what matches every object in the bucket.

Phase 6 — Resource-based policy on bucket2 (Task 6)

While still assuming BucketsAccessRole, I uploaded Image2.jpg to bucket2 and it succeeded, even though GrantBucket1Access gives the role nothing on bucket2. The reason is in bucket2's own Permissions > Bucket policy (Appendix C). It has two statements whose Principal is the

BucketsAccessRole ARN: S3Write allows s3:GetObject and s3:PutObject on bucket2/*, and ListBucket allows s3:ListBucket on the bucket.

Other panels on the same Permissions tab (Block public access, ACL, CORS, Object ownership) showed Access denied for the role, which is expected because the role has no permissions to read those settings.

Policy type	Attached to	Example in this lab
Identity-based	User, group or role	DeveloperGroupPolicy (group); GrantBucket1Access and ListAllBucketsPolicy (role)
Resource-based	The resource itself	Bucket policies on bucket2 and bucket3
Trust policy	A role	Defines who may assume the role (devuser for BucketsAccessRole)

Access to bucket1 came from the role's identity-based policy; access to bucket2 came from a resource-based policy. For a principal in the same account, one matching Allow is enough as long as nothing denies it.

Phase 7 — Challenge task: upload to bucket3

As devuser I could not upload to bucket3 and could not view its bucket policy. After assuming BucketsAccessRole I could read the policy (Appendix D). The Principal in both statements is OtherBucketAccessRole, not BucketsAccessRole. So the bucket only trusts a different role. I switched to OtherBucketAccessRole, opened bucket3, and uploaded Image2.jpg successfully.

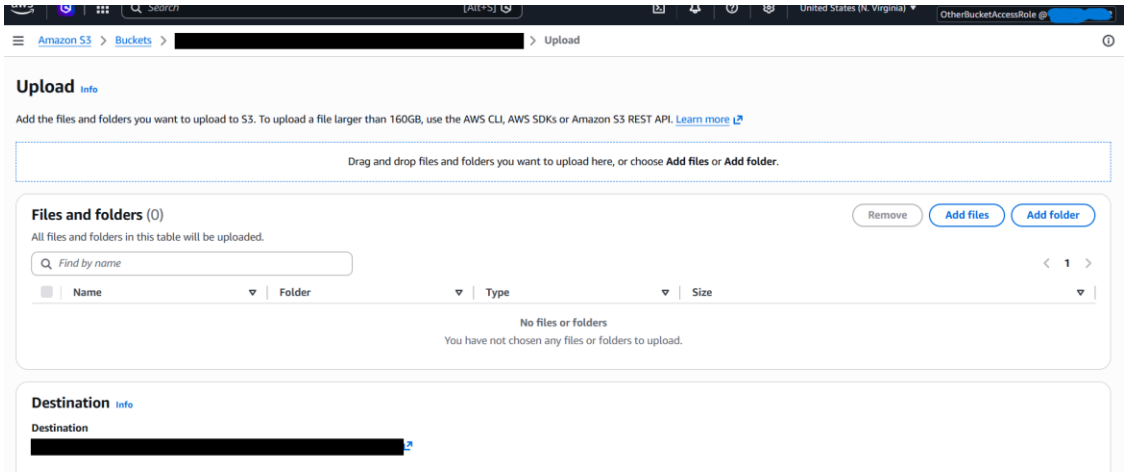


Figure 15 — Upload page for bucket3 while assuming OtherBucketAccessRole.

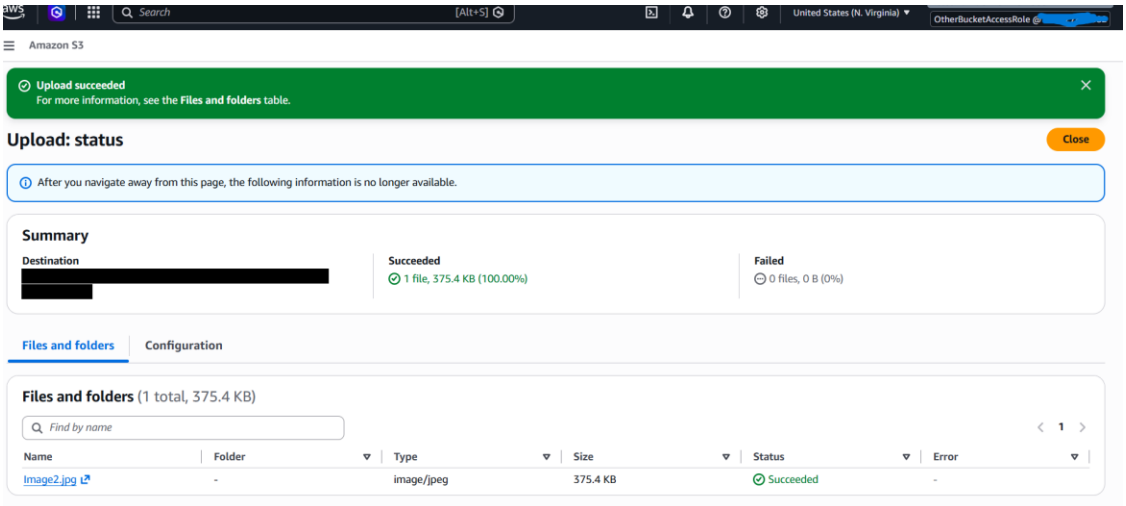


Figure 16 — Upload succeeded: 1 file, 375.4 KB.

Phase 8 — Verification and grading

#	Action	Expected	Result	Evidence
1	devuser: open EC2 Instances	Denied	Not authorized (DescribeInstances)	Fig. 2
2	devuser: key pair dropdown	Denied	Not authorized (DescribeKeyPairs)	Fig. 3
3	devuser: list buckets / open bucket1	Visible (ListBucket)	Three buckets; two objects in bucket1	Fig. 4–5
4	devuser: upload to bucket1	Denied	Access denied	Fig. 6
5	devuser: create bucket	Success	Created depi-lab3.1	Fig. 8
6	devuser: upload into own bucket	Denied	Upload failed	Fig. 9
7	devuser: download bucket1/Image2.jpg	Denied	AccessDenied (GetObject)	Fig. 10
8	BucketsAccessRole: download bucket1/Image2.jpg	Success	File downloaded	Fig. 12
9	BucketsAccessRole: IAM > User groups	Denied	Access denied to iam:ListGroups	Fig. 13
10	devuser (switched back): IAM > User groups	Success	DeveloperGroup listed	Fig. 14
11	BucketsAccessRole: upload Image2.jpg to bucket2	Success (bucket policy)	Success; confirmed by grader (Task 5 = 5/5)	Fig. 18
12	OtherBucketAccessRole: upload Image2.jpg to bucket3	Success	Upload succeeded	Fig. 16

The first submission scored 10/15 (66.67): Task 4 and the Challenge passed, but "Task 5 – Uploaded object" was 0/5 because I had downloaded Image2.jpg from bucket1 but had not uploaded it to bucket2. I assumed BucketsAccessRole again, uploaded Image2.jpg to bucket2, and resubmitted. The

grader checks the live environment, so only the missing step was needed, and the second submission scored 15/15 (100).

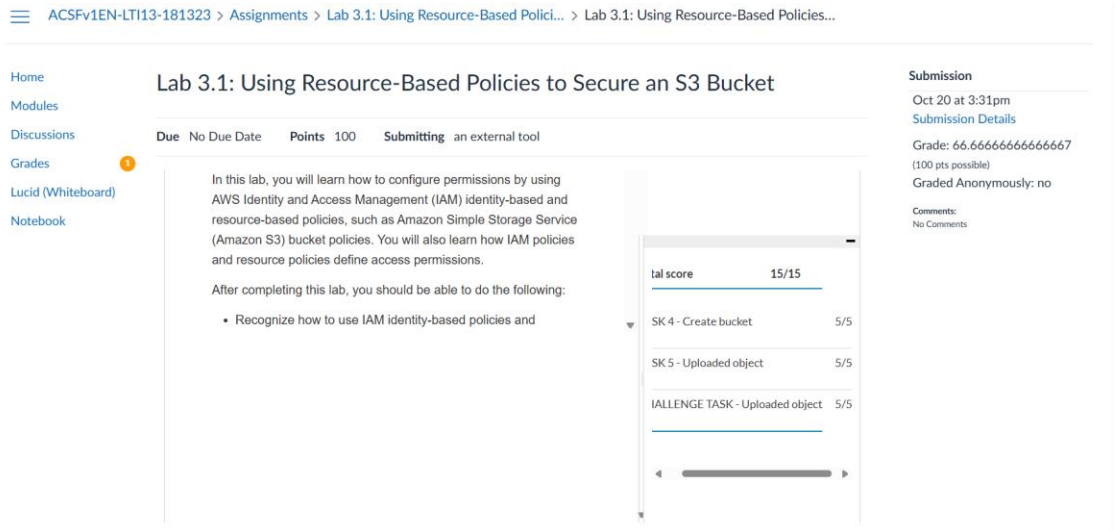


Figure 17 — First submission: 66.67, Task 5 upload missing.

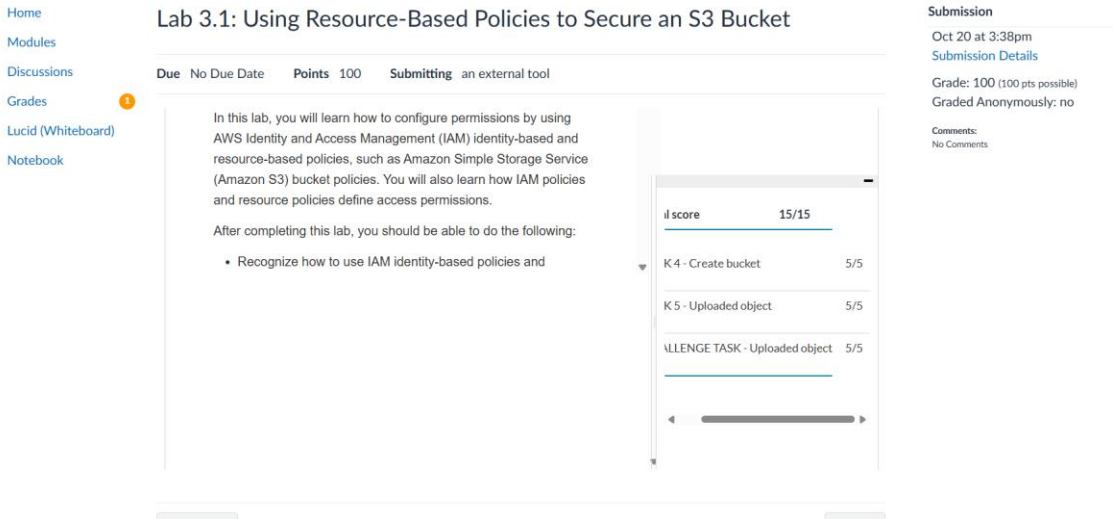


Figure 18 — Second submission: 100, all three tasks 5/5.

Key takeaways

- **Least privilege:** devuser has only what its group policy lists. Anything else is implicitly denied, and the error names the missing action.
- **Roles replace, not add:** while a role is assumed, only the role's permissions apply. That is why IAM broke in the role session and worked again after Switch back.
- **Two ways to get access:** an identity-based policy on the caller, or a resource-based policy on the resource that names the caller. bucket1 used the first, bucket2 the second.
- **Check the Principal:** bucket3 trusted OtherBucketAccessRole, not BucketsAccessRole. Reading the policy told me which role to assume.

- **Console differences:** the Access column is missing in the current S3 console, and devuser could list bucket1 contents. The policy explains both.

Not captured

I did not take screenshots of the Trust relationships tab, of the bucket2 upload, or of the bucket2/bucket3 Permissions pages. The bucket policies in the appendices were copied as text from the console, and the bucket2 upload is confirmed by the Task 5 score.

Cleanup: after grading, End Lab removes the environment.

Appendix A — DeveloperGroupPolicy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "cloudformation:Describe*",
        "cloudformation:Get*",
        "cloudformation:List*",
        "iam:Describe*",
        "iam:GetAccountAuthorizationDetails",
        "iam:GetGroup",
        "iam:GetGroupPolicy",
        "iam:GetPolicy",
        "iam:GetRole",
        "iam:GetRolePolicy",
        "iam:GetUser",
        "iam:GetUserPolicy",
        "iam:List*",
        "logs:Desc*",
        "logs:Get*",
        "logs:List*",
        "s3:CreateBucket",
        "s3:ListAllMyBuckets",
        "s3:ListBucket",
        "s3:PutAccountPublicAccessBlock",
        "s3:PutBucketOwnershipControls",
        "s3:PutBucketPublicAccessBlock",
        "sts:AssumeRole"
      ],
      "Resource": "*"
    }
  ]
}
```

Appendix B — GrantBucket1Access.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListObjects",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::<BUCKET1_NAME>",
        "arn:aws:s3:::<BUCKET1_NAME>/*"
      ]
    }
  ]
}
```

Appendix C — bucket2 bucket policy

```
{
  "Version": "2008-10-17",
  "Statement": [
    {
      "Sid": "S3Write",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::<ACCOUNT_ID>:role/BucketsAccessRole"
      },
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::<BUCKET2_NAME>/*"
    },
    {
      "Sid": "ListBucket",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::<ACCOUNT_ID>:role/BucketsAccessRole"
      },
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::<BUCKET2_NAME>"
    }
  ]
}
```

Appendix D — bucket3 bucket policy

```
{
  "Version": "2008-10-17",
  "Statement": [
    {
      "Sid": "S3Write",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::<ACCOUNT_ID>:role/OtherBucketAccessRole"
      },
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::<BUCKET3_NAME>/*"
    },
    {
      "Sid": "ListBucket",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::<ACCOUNT_ID>:role/OtherBucketAccessRole"
      },
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::<BUCKET3_NAME>"
    }
  ]
}
```